

Generation

Define los Criterios para la Tarea de la IA y Escribe los Prompts

Paso 1: Definir los Criterios Específicos para la Tarea (Marco CLEAR)

Utiliza el marco CLEAR para definir los criterios específicos que tu herramienta de IA debe seguir al analizar los archivos de registro.

1. Relevancia del Contexto

- ¿Qué alcance debe tener la IA? ¿Hay tipos específicos de registros o palabras clave que se deben priorizar?
- Ejemplo: "Enfocarse en errores y advertencias relacionadas con la actividad del usuario; excluye los registros informativos a menos que estén marcados como críticos."

La herramienta de IA debe centrarse en analizar archivos de registro o logs provenientes de aplicaciones, servidores, APIs, bases de datos y otros sistemas informáticos.

Debe priorizar especialmente:

- Errores.
- Advertencias.
- Eventos críticos.
- Problemas recurrentes.
- Mensajes de error únicos.
- Patrones y tendencias.
- Fallos relacionados con bases de datos, red, autenticación, memoria, APIs y servidores.

Los registros informativos deben utilizarse principalmente cuando ayuden a comprender el contexto de un error o permitan detectar una relación entre diferentes eventos.

La IA también debe utilizar las fechas y horas presentes en los registros para identificar cuándo comienza un problema, cuándo termina y si existen periodos donde aumenta la cantidad de incidencias.

2. Coincidencia de Tono y Estilo del Lenguaje

- *¿Qué tono debe usar la IA? (por ejemplo, técnico, amigable)*

La IA debe utilizar un tono técnico, profesional, claro y objetivo.

Las respuestas deben ser fáciles de comprender y evitar tecnicismos innecesarios. Cuando sea necesario utilizar términos técnicos, deben presentarse de manera comprensible.

Los resultados deben organizarse utilizando:

- Resúmenes.
- Tablas.
- Listas.
- Estadísticas.
- Recomendaciones.

La información más importante debe aparecer primero para facilitar la identificación rápida de problemas.

3. Manejo de Errores y Escalamiento

- *¿Cómo debe responder la IA si hay incertidumbre o falta de datos?*

Cuando la IA no tenga suficiente información para determinar la causa de un problema, deberá indicarlo claramente.

Debe diferenciar entre:

- Hechos confirmados en los registros.
- Patrones observados.
- Posibles causas o hipótesis.

La IA nunca debe presentar una hipótesis como un hecho confirmado.

Si detecta un error crítico, recurrente o que pueda afectar seriamente la disponibilidad del sistema, deberá marcarlo como prioritario y recomendar su revisión por parte del equipo técnico.

Cuando los registros estén incompletos o no proporcionen suficiente contexto, la herramienta deberá indicar qué información adicional podría ayudar a continuar con el diagnóstico.

4. Precisión y Fiabilidad de las Respuestas

- *¿Cómo asegurarás que las respuestas de la IA sean confiables y basadas en datos?*

Todas las conclusiones deberán estar basadas exclusivamente en la información disponible en los registros.

La herramienta no deberá inventar:

- Errores.
- Fechas.
- Cantidades.
- Eventos.
- Causas.
- Relaciones entre eventos.

Los mensajes repetidos deberán agruparse correctamente y se deberá indicar cuántas veces aparece cada problema.

También deberá proteger información sensible como:

- Contraseñas.
- Tokens.
- Claves API.
- Cookies.
- Credenciales.

Cuando se presente una posible causa, deberá identificarse claramente como una hipótesis.

5. Eficiencia de las Respuestas

- *¿Qué tipos de preguntas debe ser capaz de manejar la IA? ¿Cómo debe adaptar sus respuestas según la complejidad de la pregunta?*

5. Eficiencia de las Respuestas

La herramienta debe ser capaz de analizar diferentes tipos y cantidades de registros.

Debe poder responder preguntas como:

- ¿Cuántos errores existen?
- ¿Cuáles son los errores más frecuentes?
- ¿Cuántas advertencias aparecen?
- ¿Cuáles son los mensajes de error únicos?
- ¿Qué problemas se repiten?
- ¿Existen patrones?
- ¿Hay alguna tendencia temporal?
- ¿Qué problema debería investigarse primero?
- ¿Existen eventos relacionados?

Cuando los registros sean extensos, la IA deberá agrupar eventos similares y evitar mostrar información repetitiva innecesaria.

Para preguntas simples deberá responder de manera breve y directa. Para análisis más complejos deberá proporcionar estadísticas, tablas, patrones, posibles causas y recomendaciones.

Paso 2: Escribir los Prompts (Marco TRACI)

Usar el marco TRACI para redactar 1-3 prompts que dirijan la herramienta de IA a realizar cada tarea.

- **Tarea 1: Resumir los registros**

Prompt 1

Analiza los siguientes registros como un especialista técnico en análisis de logs. Identifica errores, advertencias y eventos críticos. Genera un resumen breve del estado general del sistema y muestra la cantidad de incidencias encontradas. La respuesta está dirigida a un supervisor de TI que necesita comprender rápidamente los principales problemas. Basa todas tus conclusiones únicamente en la información disponible en los registros.

Prompt 2

Actúa como un analista de sistemas y revisa los registros proporcionados. Crea un resumen estructurado que incluya el número de errores, advertencias, eventos críticos y mensajes de error únicos. Presenta primero los problemas de mayor importancia y evita repetir mensajes idénticos.

Prompt 3

Resume los registros proporcionados de manera clara y concisa. Identifica los eventos más importantes y crea una tabla con las columnas: tipo de evento, mensaje, número de ocurrencias, primera aparición, última aparición y prioridad.

- **Tarea 2: Identificar problemas recurrentes**

Prompt 1

Actúa como un analista de soporte técnico. Examina los registros e identifica los errores y advertencias que aparecen repetidamente. Agrupa los mensajes iguales o similares, cuenta sus ocurrencias y crea un ranking de los cinco problemas más frecuentes

Prompt 2

Analiza los registros y crea una lista de todos los mensajes de error únicos. Para cada error indica cuántas veces aparece y clasifica su prioridad como crítica, alta, media o baja.

Prompt 3

Identifica los problemas recurrentes presentes en estos registros. Ordena los resultados de mayor a menor frecuencia y explica cuáles deberían investigarse primero considerando tanto la cantidad de ocurrencias como su gravedad.

- **Tarea 3: Detectar patrones y tendencias**

Prompt 1

Actúa como un experto en monitoreo y diagnóstico de sistemas. Analiza cronológicamente los registros y detecta patrones, tendencias y posibles relaciones entre diferentes eventos. Identifica horarios con mayor número de errores y problemas que aparezcan después de otros eventos.

Prompt 2

Busca patrones en los siguientes registros. Identifica errores repetitivos, componentes afectados y secuencias de eventos que puedan estar relacionadas. Distingue claramente entre hechos confirmados y posibles hipótesis.

Prompt 3

Analiza estos registros para detectar posibles problemas sistémicos. Identifica tendencias, relaciones temporales y eventos recurrentes. Finaliza con una lista de los problemas que deberían investigarse primero y explica brevemente por qué.

Plantilla de Documentación

Instrucciones:

1. Introducción

- **Propósito de la Guía**

El propósito de esta guía es documentar el diseño, configuración y funcionamiento de una herramienta personalizada de inteligencia artificial creada en TextCortex para automatizar el análisis de registros o logs.

La herramienta busca reducir el tiempo necesario para revisar manualmente grandes cantidades de registros y facilitar la identificación de errores, advertencias, problemas recurrentes y posibles tendencias.

Alcance

La herramienta está diseñada para analizar registros provenientes de aplicaciones, servidores, APIs, bases de datos y otros sistemas informáticos.

Puede:

- Resumir registros.
- Detectar errores.
- Identificar advertencias.
- Reconocer eventos críticos.
- Agrupar mensajes repetidos.
- Contar incidencias.
- Identificar errores únicos.
- Detectar patrones y tendencias.
- Generar posibles hipótesis.
- Recomendar problemas a investigar.

La herramienta no sustituye completamente el diagnóstico realizado por un especialista y no debe asumir causas que no puedan demostrarse mediante los registros.

- **Alcance**

Analiza los registros y crea una lista de todos los mensajes de error únicos. Para cada error indica cuántas veces aparece y clasifica su prioridad como crítica, alta, media o baja.

2. Descripción de la Herramienta

- **Descripción**

La herramienta fue creada utilizando TextCortex y se configuró como un asistente especializado en análisis de registros denominado **LogAnalyzer AI**.

Su función es recibir archivos o fragmentos de logs y convertir grandes cantidades de información técnica en resultados estructurados y fáciles de interpretar.

El agente utiliza un tono técnico, profesional y claro. También cuenta con reglas que determinan qué debe hacer siempre y qué comportamientos debe evitar.

- **Características Principales**

- Análisis automático de registros.
- Identificación de errores y advertencias.
- Detección de eventos críticos.
- Conteo de incidencias.
- Agrupación de errores repetidos.
- Identificación de mensajes únicos.
- Clasificación de problemas por prioridad.
- Detección de patrones.
- Análisis de tendencias temporales.
- Identificación de posibles relaciones entre eventos.
- Generación de recomendaciones.
- Protección de información sensible.
- Presentación de resultados mediante tablas y resúmenes.

3. Proceso de Desarrollo

Guía Paso a Paso

1. Establecimiento de Objetivos

Primero se definió que el objetivo principal de la herramienta sería automatizar y agilizar el análisis de registros.

Los objetivos específicos fueron:

- Resumir registros.
- Identificar errores y advertencias.
- Detectar mensajes únicos.
- Contar problemas recurrentes.
- Detectar patrones.
- Priorizar incidencias.

2. Recopilación de Datos

Se utilizaron registros de muestra que contienen diferentes tipos de eventos, como:

- INFO.
- WARNING.
- ERROR.
- CRITICAL.

También se consideraron ejemplos de fallos relacionados con:

- Bases de datos.
- APIs.
- Autenticación.
- Memoria.
- Red.
- Servidores.

3. Procesamiento previo de Datos

Antes del análisis se definieron criterios para que la herramienta pudiera:

- Reconocer diferentes tipos de eventos.
- Agrupar mensajes repetidos.
- Utilizar marcas de tiempo.
- Diferenciar errores únicos.
- Ordenar problemas por prioridad.
- Evitar información redundante.

También se establecieron medidas para evitar la exposición de información sensible.

4. Entrenamiento y Refinamiento de la Herramienta

En TextCortex se configuraron diferentes elementos del agente:

Identidad: se definió como un asistente especializado en análisis de registros.

Personalidad: se configuró para utilizar un tono técnico, profesional, claro y objetivo.

Mensaje de bienvenida: se creó una introducción explicando al usuario que puede proporcionar archivos o fragmentos de registros.

Reglas Always: se estableció que siempre debe resumir, contar errores, agrupar problemas, identificar patrones y separar hechos de hipótesis.

Reglas Never: se estableció que nunca debe inventar errores, cantidades o causas ni presentar hipótesis como hechos.

También se creó una plantilla reutilizable para introducir registros, indicar el objetivo del análisis y seleccionar el nivel de detalle de la respuesta.

5. Pruebas y Evaluación

La herramienta se probó utilizando registros de muestra con errores intencionales.

Por ejemplo:

```
2026-10-05 08:00:01 INFO Server started successfully
2026-10-05 08:05:12 WARNING API response time exceeded 2000ms
2026-10-05 08:06:43 ERROR Database connection failed
2026-10-05 08:07:01 ERROR Database connection failed
2026-10-05 08:10:22 WARNING API response time exceeded 2000ms
2026-10-05 08:12:18 ERROR User authentication failed
2026-10-05 08:15:44 ERROR Database connection failed
2026-10-05 08:17:31 WARNING High memory usage detected
2026-10-05 08:20:12 ERROR Database connection failed
2026-10-05 08:21:05 CRITICAL Database service unavailable
```

La herramienta debía identificar correctamente que el mensaje **Database connection failed** aparece cuatro veces y reconocer posteriormente un evento crítico relacionado con el servicio de base de datos.

También debía detectar que existe una posible relación entre los errores recurrentes de conexión y la posterior indisponibilidad del servicio, sin afirmar automáticamente que uno provocó al otro.

4. Principales Desafíos y Soluciones

- **Desafío 1**

Uno de los principales desafíos es analizar grandes cantidades de registros sin perder información importante.

Un archivo puede contener cientos o miles de eventos, muchos de ellos repetidos.

Solución:

Configurar la herramienta para agrupar eventos similares, contar sus ocurrencias y presentar solamente los elementos relevantes

- **Desafío 2**

Otro desafío consiste en evitar que la inteligencia artificial interprete una relación temporal como una relación de causa y efecto.

Por ejemplo, dos errores pueden aparecer uno después del otro sin que necesariamente uno haya provocado al otro.

Solución:

Establecer reglas claras para diferenciar:

- Hechos confirmados.
- Patrones detectados.
- Posibles hipótesis.

La herramienta debe indicar cuando no existe suficiente información para confirmar una causa.

- **Consejos para el Éxito**
 - **Utilizar registros suficientemente completos.**
 - **Mantener formatos de fecha y hora consistentes.**
 - **Proporcionar contexto cuando sea necesario.**
 - **Revisar los errores clasificados como críticos.**
 - **Utilizar prompts específicos.**
 - **No utilizar exclusivamente la cantidad de ocurrencias para determinar la gravedad.**
 - **Refinar las reglas conforme se realicen nuevas pruebas.**

5. Casos de Uso Potenciales

- **Más Allá del Análisis de Registros**

La herramienta también podría utilizarse para:

- Analizar reportes de incidencias.
- Clasificar tickets de soporte.
- Analizar errores de aplicaciones.
- Revisar respuestas de APIs.
- Analizar eventos de seguridad.

- Detectar intentos repetidos de autenticación.
- Resumir reportes técnicos.
- Analizar alertas provenientes de sistemas de monitoreo.

- **Oportunidades Futuras**

En el futuro, la herramienta podría integrarse con sistemas de monitoreo para recibir información automáticamente.

También podría:

- Crear reportes automáticos.
- Detectar anomalías.
- Comparar registros entre distintos periodos.
- Generar gráficas de incidencias.
- Clasificar problemas automáticamente.
- Sugerir acciones de solución.
- Generar alertas cuando detecte problemas críticos.

6. Instrucciones para el Usuario

- **Cómo Usar la Herramienta**

1. Abrir LogAnalyzer AI en TextCortex.
2. Proporcionar un archivo de registros o pegar directamente los registros en el chat.
3. Especificar el objetivo del análisis.
4. Indicar el nivel de detalle deseado.
5. Ejecutar el análisis.
6. Revisar el resumen general.
7. Consultar los errores y advertencias encontrados.
8. Revisar los patrones y tendencias.
9. Analizar las recomendaciones proporcionadas.

- **Entradas y Salidas**

Entradas:

La herramienta puede recibir:

- Archivos de logs.
- Fragmentos de registros.
- Registros de aplicaciones.
- Registros de servidores.
- Eventos de APIs.
- Registros de bases de datos.

Salidas:

La herramienta puede generar:

- Resumen general.
- Estadísticas.
- Lista de errores.
- Lista de advertencias.
- Errores únicos.
- Conteo de problemas recurrentes.
- Ranking de incidencias.
- Patrones.
- Tendencias.
- Posibles causas.
- Recomendaciones.

- **Consejos para el Usuario**

Para obtener mejores resultados:

- Proporcionar registros completos cuando sea posible.
 - Mantener las marcas de tiempo.
 - Evitar eliminar información que pueda proporcionar contexto.
 - Especificar claramente qué se desea investigar.
 - Utilizar un nivel de detalle apropiado.
 - Recordar que las posibles causas generadas por la herramienta deben validarse antes de tomar decisiones importantes.
-

7. Escenarios de Ejemplo

- **Escenario 1**

Un servidor comienza a registrar repetidamente el mensaje:

ERROR Database connection failed

Después de varias apariciones se registra:

CRITICAL Database service unavailable

El usuario proporciona los registros a LogAnalyzer AI.

La herramienta identifica que el error de conexión aparece varias veces, muestra el número de ocurrencias y señala posteriormente el evento crítico.

El sistema detecta una posible tendencia relacionada con la disponibilidad de la base de datos y recomienda revisar primero el servicio de base de datos, sus conexiones y su estado.

La herramienta presenta esta relación como una posible hipótesis y no como una causa confirmada.

- **Escenario 2**

Los registros muestran repetidamente:

WARNING API response time exceeded 2000ms

Posteriormente aparecen diferentes errores relacionados con solicitudes de la aplicación.

LogAnalyzer AI agrupa las advertencias, cuenta sus ocurrencias y utiliza las marcas de tiempo para determinar cuándo comenzaron a aumentar.

La herramienta identifica un posible problema de rendimiento y recomienda investigar el tiempo de respuesta de la API, los servicios relacionados y los recursos del servidor.

De esta manera, el usuario puede identificar rápidamente una tendencia que sería más difícil de detectar revisando manualmente cada línea del registro.

Reflexión Final

Uno de los aspectos más desafiantes del análisis de registros con inteligencia artificial es diferenciar entre eventos aislados, problemas recurrentes y problemas realmente críticos.

También representa un desafío identificar relaciones entre eventos sin asumir incorrectamente que existe causalidad.

Los criterios definidos mediante el marco CLEAR ayudan a mantener el análisis relevante, preciso, confiable y eficiente.

Por otra parte, los prompts creados mediante el marco TRACI permiten proporcionar instrucciones específicas según la tarea que se desea realizar.

La combinación de ambos marcos permite que LogAnalyzer AI produzca resultados más consistentes, organizados y prácticos, facilitando la identificación de problemas y reduciendo el tiempo requerido para analizar grandes cantidades de registros.